



UNIVERSIDAD NACIONAL DE LOJA

Carrera de Computación

ESTRUCTURA DE DATOS

APE02

Análisis Matemático de Rendimiento y Pregunta de Control N.º 4

Estudiante: Jahir Campoverde

Docente: Cristian Ramiro Narváez Guillén

Fecha: 8 de octubre de 2026

1. Análisis Matemático de Rendimiento (ABI)

1.1. Complejidad espacial

En esta práctica se comparan dos métodos de lectura de un archivo CSV que contiene un millón de coordenadas geográficas.

La lectura síncrona mediante `readFileSync()` carga todo el archivo en memoria RAM. Por ello, su consumo de memoria aumenta proporcionalmente al tamaño de los datos:

$$\boxed{S_{\text{sync}}(n) = O(n)} \quad (1)$$

En cambio, la lectura mediante `createReadStream()` procesa los datos por fragmentos (*chunks*), sin almacenar todo el archivo.

Con un búfer de tamaño fijo, su complejidad espacial auxiliar es:

$$\boxed{S_{\text{stream}}(n) = O(1)} \quad (2)$$

Ambos métodos requieren recorrer el archivo, por lo que presentan una complejidad temporal lineal:

$$T(n) = O(n) \quad (3)$$

1.2. Funcionamiento de libuv

Node.js utiliza la biblioteca `libuv` para gestionar operaciones asíncronas de entrada y salida.

El sistema operativo coordina el acceso al disco y la transferencia de datos mediante las interfaces y buses del hardware.

En las operaciones de archivos, `libuv` puede utilizar un *Thread Pool*, mientras que el *Event Loop* coordina los eventos en JavaScript.

Esto permite procesar archivos mediante Streams sin bloquear el hilo principal durante la espera de I/O.

1.3. Relación con Green Computing

El uso de Streams permite reducir la cantidad de memoria necesaria para procesar archivos grandes.

Al evitar almacenar todos los registros, también puede disminuir la presión sobre el recolector de basura (*Garbage Collector*).

Esto favorece el uso eficiente de los recursos computacionales, aunque el ahorro energético depende también del tiempo de ejecución y de la actividad del hardware.

2. Pregunta de Control N.º 4: Causa Final

Pregunta

Si todo dato en ejecución debe residir en la memoria principal para ser computado por la CPU, ¿cuál es el propósito (*telos*) de la persistencia en el disco físico, considerando que las operaciones de I/O son más lentas que el acceso a memoria caché?

Respuesta

Desde la perspectiva de la causa final aristotélica, el propósito de la persistencia es garantizar que la información se conserve incluso después de finalizar un programa o apagar el computador.

Aunque la memoria RAM y la memoria caché ofrecen mayor velocidad, son volátiles. Por esta razón, el almacenamiento secundario permite conservar los datos de manera duradera.

En la práctica APE02, el millón de coordenadas permanece almacenado en un archivo CSV, mientras que Node.js utiliza Streams para procesarlo por fragmentos sin cargarlo completamente en memoria.

Por lo tanto, la finalidad de la persistencia no es proporcionar la máxima velocidad, sino asegurar la conservación, disponibilidad y recuperación futura de los datos.

3. Conclusión

La lectura mediante Streams permite procesar grandes archivos con una cantidad de memoria auxiliar acotada, mientras que la lectura síncrona requiere memoria proporcional al tamaño del archivo.

Asimismo, la persistencia garantiza la conservación de la información, complementando la eficiencia del procesamiento en memoria.